

Szoftvertechnológia és -technikák

10. Előadás

*A szoftverfejlesztés fázisai, dokumentáció,
tesztelés alapjai*

A szoftverfejlesztés fázisai

It's alive!

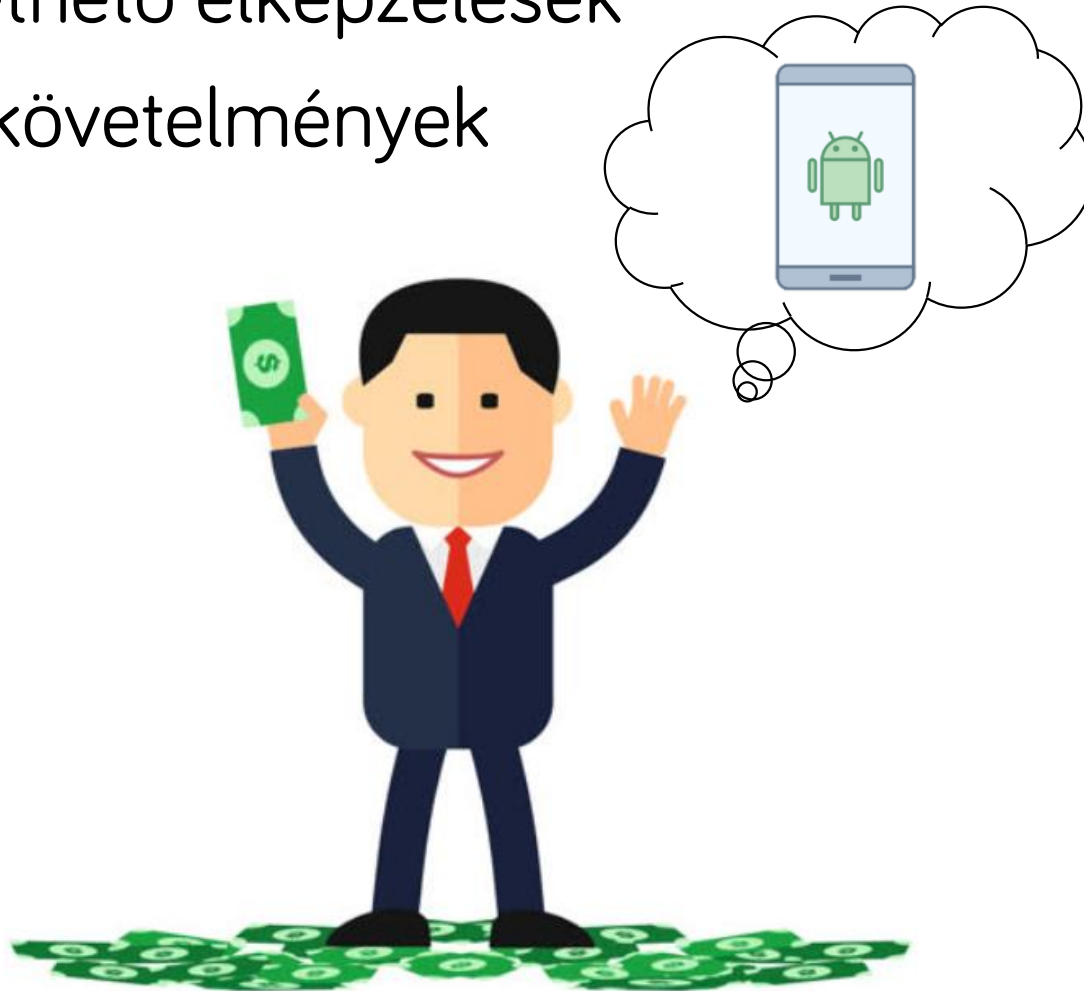


Ideális világ



Ideális világ - Megrendelő

- Jól követhető elképzelések
- Pontos követelmények



Ideális világ - Fejlesztés

- Könnyen, tisztán megértjük a feladatot
- Mehet a fejlesztés!



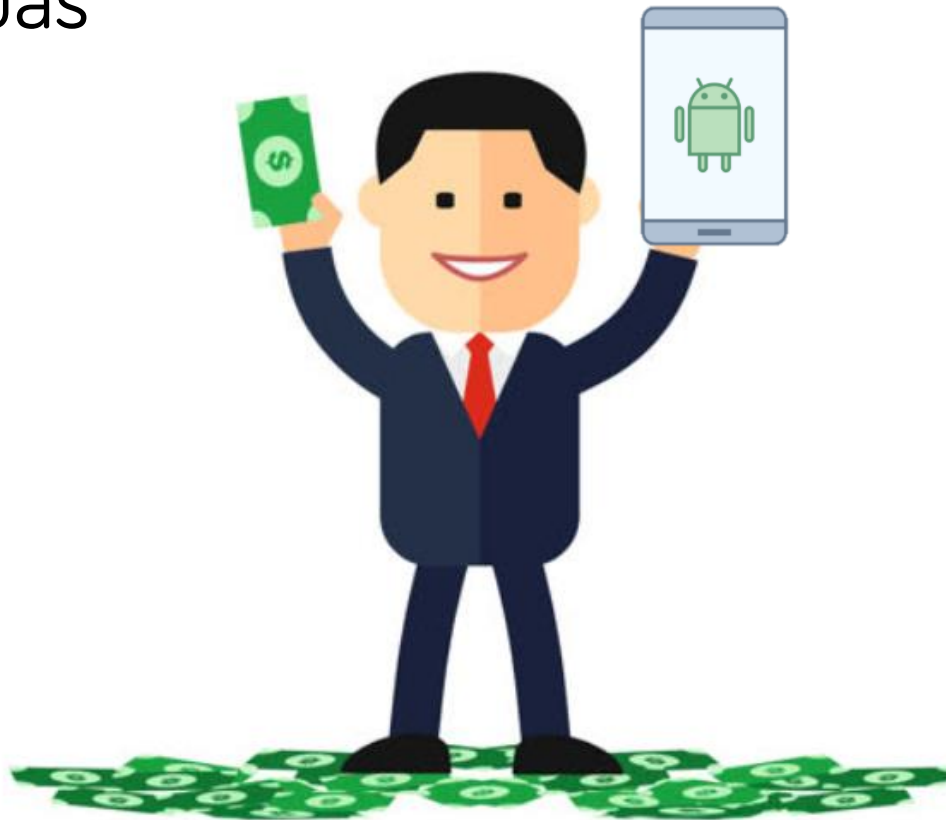
Ideális világ – Kész termék

- Elkészül az alkalmazás határidőre



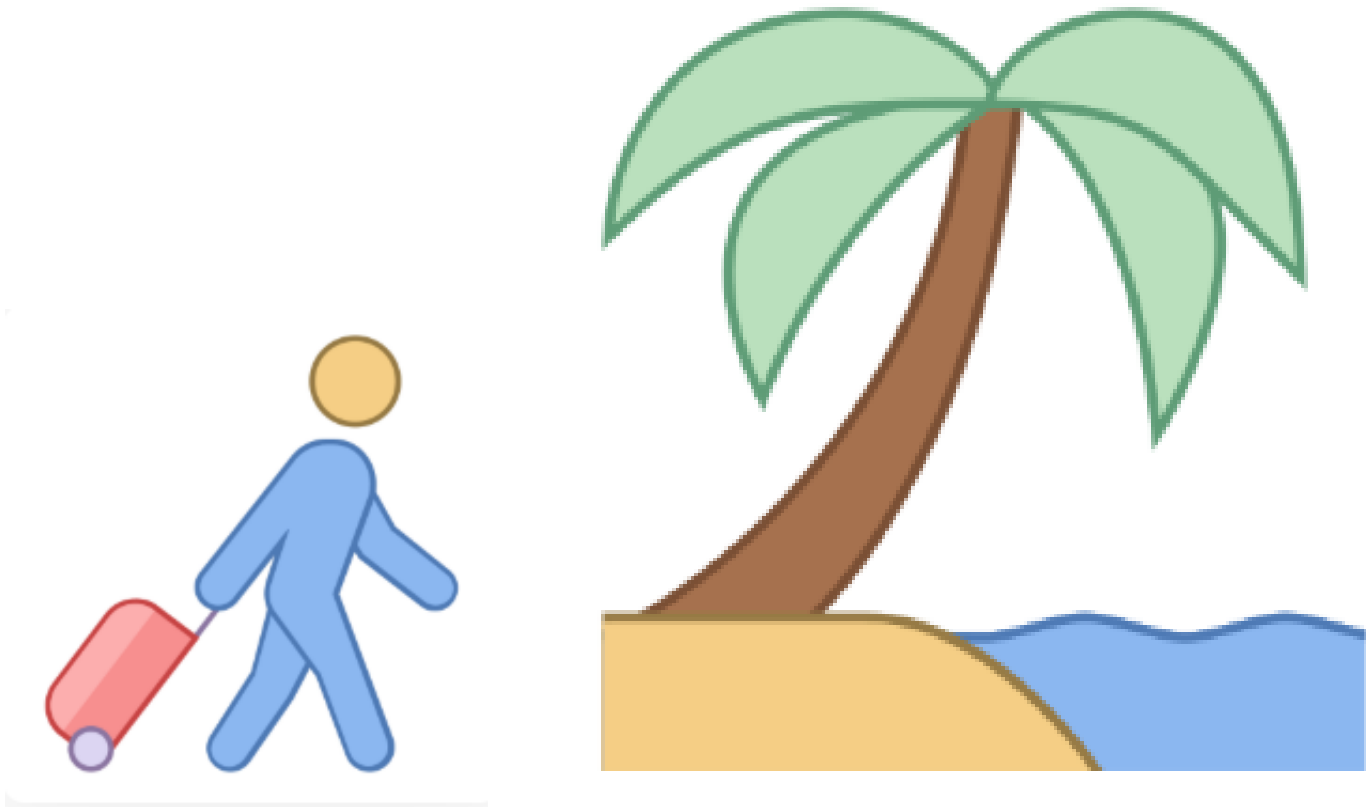
Ideális világ - Átadás

- Elégedett megrendelő
- Könnyű átadás

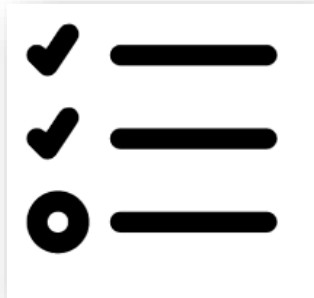


Ideális világ

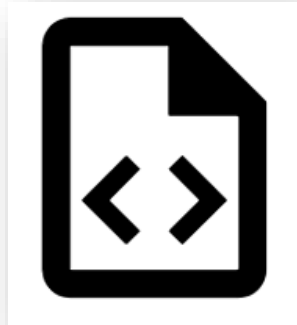
- Mehetünk nyaralni...



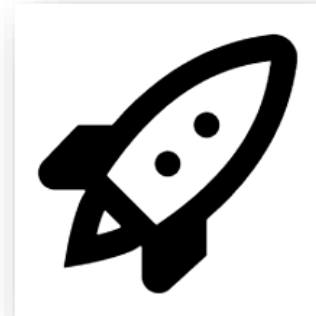
Ideális világban a szoftverfejlesztés fázisai



TERVEZÉS



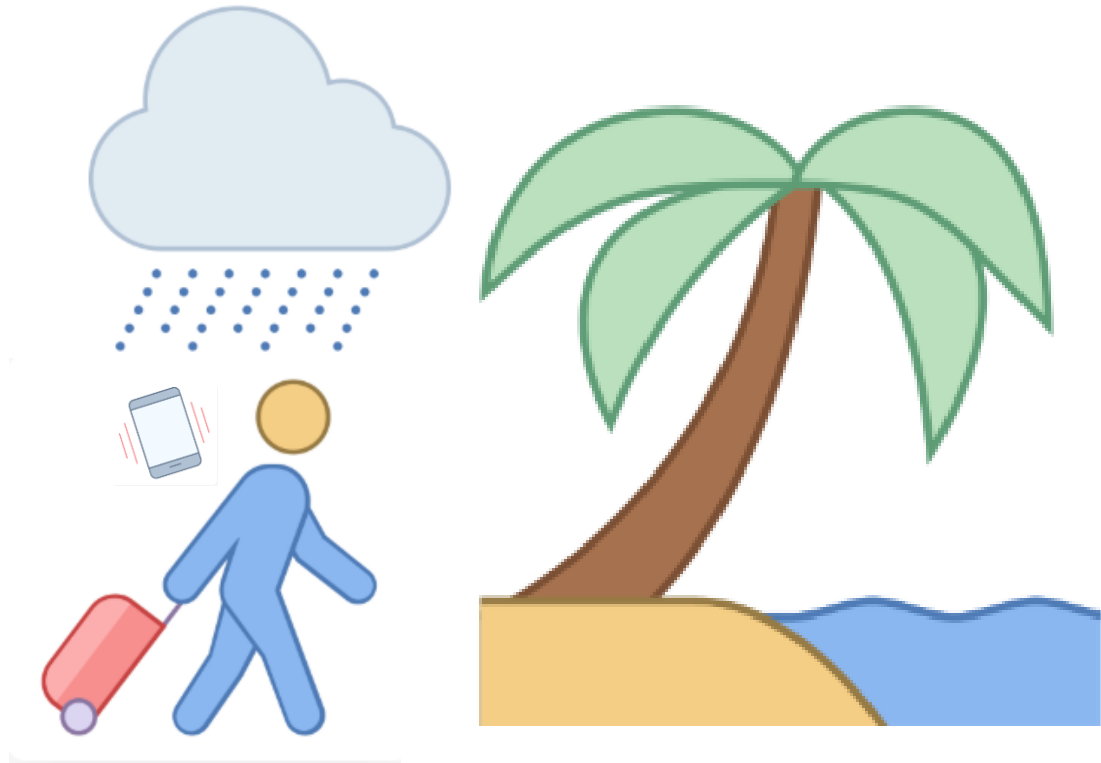
FEJLESZTÉS



TELEPÍTÉS

A valóság ennél árnyaltabb...

- Áramszünet után fura hiba



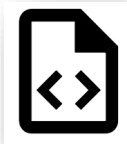
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



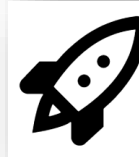
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS

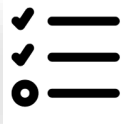


ÜZEMELTETÉS, KARBANTARTÁS

A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



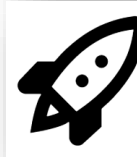
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Követelmények definiálása

- A Projekt fogalma:
 - > **egyedi jellemzőkkel rendelkező rendszer** kidolgozására vállalkozik
 - > különféle **koordinált tevékenységek** alkotják
 - > meghatározott a **célja**
 - > jól definiált kezdő és befejező **dátummal** rendelkezik
 - > **átmeneti** vállalkozás (a projekt végeztével a kialakított szervezeti és strukturális felépítés megszűnik vagy átalakul)
 - > egyéb keretek (például költség, információ biztonság stb.)

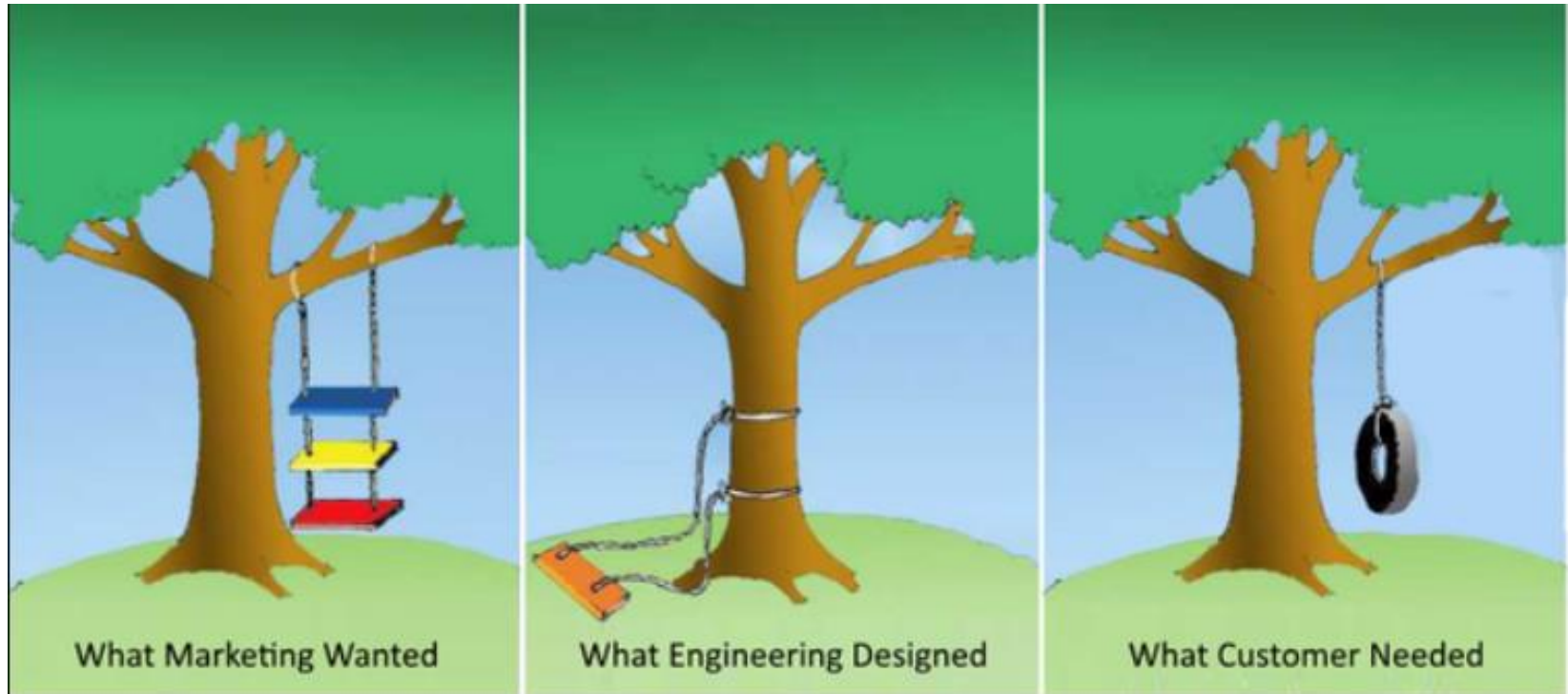
Követelmények definiálása

- Meghatározó tényezők: **idő, költség, teljesítmény és minőség**
- Példa: A megrendelő nem bíz a műholdakban, és szeretne egy saját navigációs, útvonal optimalizációs rendszert, ezzel bíz meg minket (gondoljunk a Waze-re)

Követelmények definiálása

- Mikorra szeretné ezt a megrendelő? Milyen határidővel?
 - > 2 év múlva
- Milyen platformra szeretné a megrendelő?
 - > Android, iOS? Mindkettő?
- Mit kell pontosan tudni az appnak?
 - > Milyen pontossággal működjön?
 - > Milyen gyorsan?
 - > Milyen hibahatárral?
 - > Milyen funkció vannak pontosan?

Követelmények definiálása



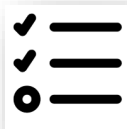
Követelmények rögzítése

- A követelmények részletei meghatározzák a költségeket (és átfutást)
- A részletek adják a projekt funkcionális kereteit
 - a szerződés kereteit
 - a change requestek alapját

A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



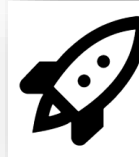
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Szoftvertervezés

- A szoftvertervezési folyamatnak kell biztosítania, hogy a fejlesztés végén egy „jó szoftver” szülessen
- A „jó szoftver” egy megfelelő kompromisszum a különféle elvárások célkeresztjében
 - > A „jó szoftver” ritkán az elképzelhető legjobb...



Milyen a jó szoftver?

- Funkcionálisan teljes
 - > Ellátja a kitűzött funkcióit
- Robosztus
 - > Hibatűrő (felismeri és orvosolja a hibát)
 - > Jól viseli a változtatásokat
- Mérhető
 - > Futásidő (pl. UI betöltése)
- Debugolható:
 - > Pl. logok
 - > Monitorozható

A jó szoftver

- Karbantartható
 - > Architektúra, kommentek, dokumentáció
- Megbízható
- Újra felhasználható
- Tovább fejleszthető

Szoftvertervezés

- A követelmények alapján elkészül a szoftverterv
- Kialakul a rendszer architektúrája:
 - > Milyen egységek vannak?
 - > Ezek hogyan kapcsolódnak
- A szoftvertervezés eredményei modellek és dokumentáció (terméktípustól függően más és más jellegű és mennyiségű)

Minőség és költségek

- A felhasználó néha nem fizeti meg a lehető legjobb minőséget → ???
- Valóság: egyszerűsítünk, butítunk, ...
- Példa:
 - > Jobbgombos menüt sok idő lenne ide betenni
 - > Jó lesz, ha lenyomja az „n” betűt, létrejön az új elem
 - Beleírjuk egy tooltipbe és a kézikönyvbe

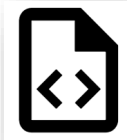
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



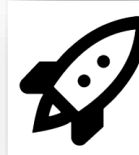
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Implementálás

- A szoftvertervezési fázisban készített szoftverterv ebben a szakaszban valósul meg
- A funkciók itt valósulnak meg
- Általában az aktív fejlesztés leghosszabb szakasza

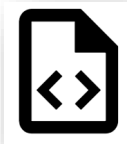
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



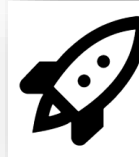
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Hibakeresés

- A “tökéletes” és “hibátlan” szoftver legendája
- Tévedni emberi
 - > Kapkodás
 - > Monotonitás
 - > Fáradtság
- Hibatípusok
 - > Implementációs hiba (“if (a=5) ...”)
 - > Kommunikációs hiba
 - > Rossz kódolás, verziókezelés, ...

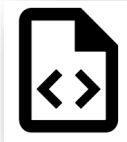
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



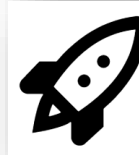
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Tesztelés

- Tény: minden szoftverben vannak hibák
- A minőség ellenőrzők megvizsgálják a szoftvert, hogy a specifikáció szerint működik-e?
- A hibák visszajutnak a fejlesztőkhöz és a tesztelés előlről indul
- Gyakran használunk átmeneti állapotot: béta verzió, meghívott felhasználók stb.

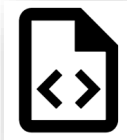
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



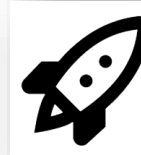
IMPLEMENTÁLÁS



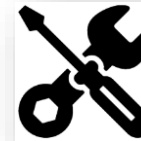
HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Telepítés (deployment)

- Telepítés, terjesztés
- Más környezet – más működés
- Pontosán ugyanaz a verzió menjen ki, mint amit a tesztelők jóváhagytak!

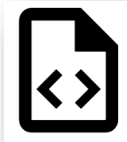
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



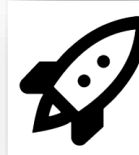
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS,
KARBANTARTÁS

Üzemeltetés és Karbantartás

- A szoftverélelciklusának általában a leghosszabb szakasza
 - > Gondoljunk az újonnan vett és a használt autókra! Mennyi költség megy el az autók szervizelésére, karbantartására!

Üzemeltetés és Karbantartás

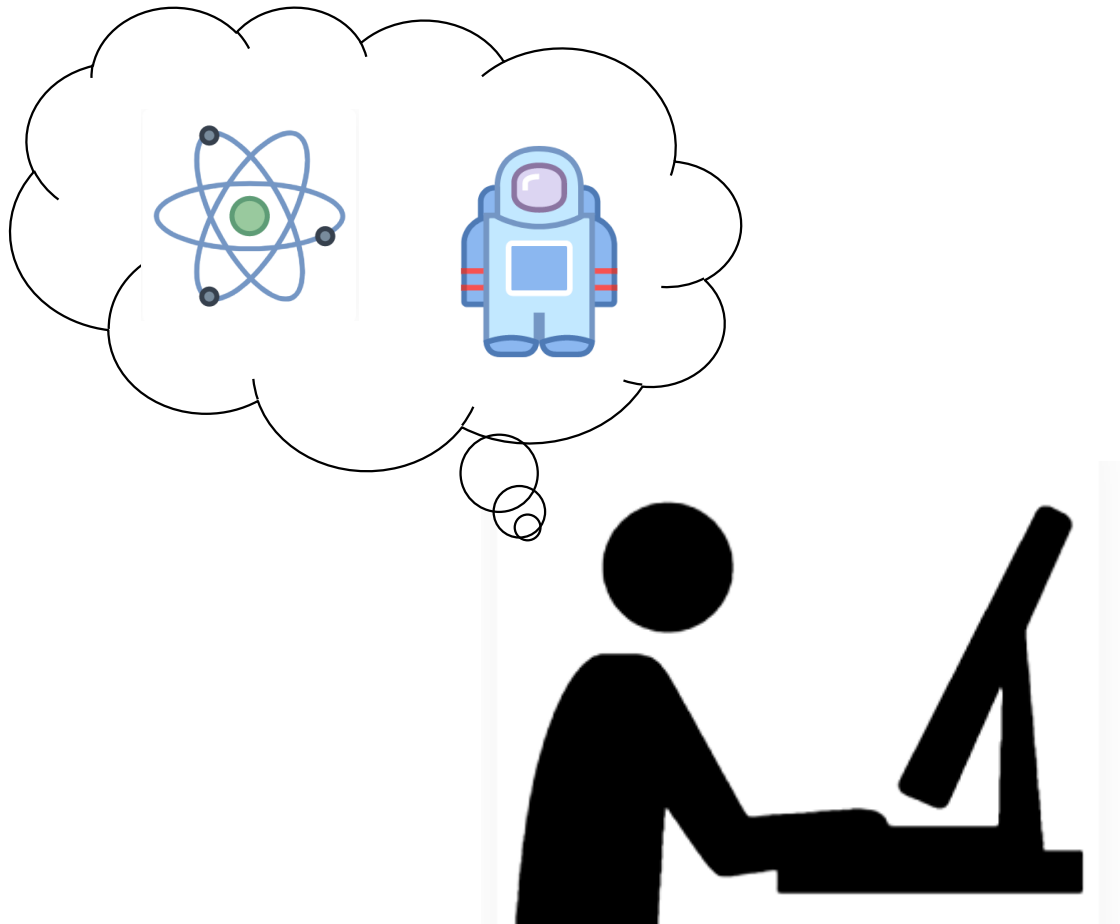
- Továbbfejlesztési lehetőségek
 - > Implementáció/Architektúra fejlesztése
 - > Új követelmények fellépése
- A hibák javítása
- Megváltozott környezethez illesztés

Eredmény a fázisok végén

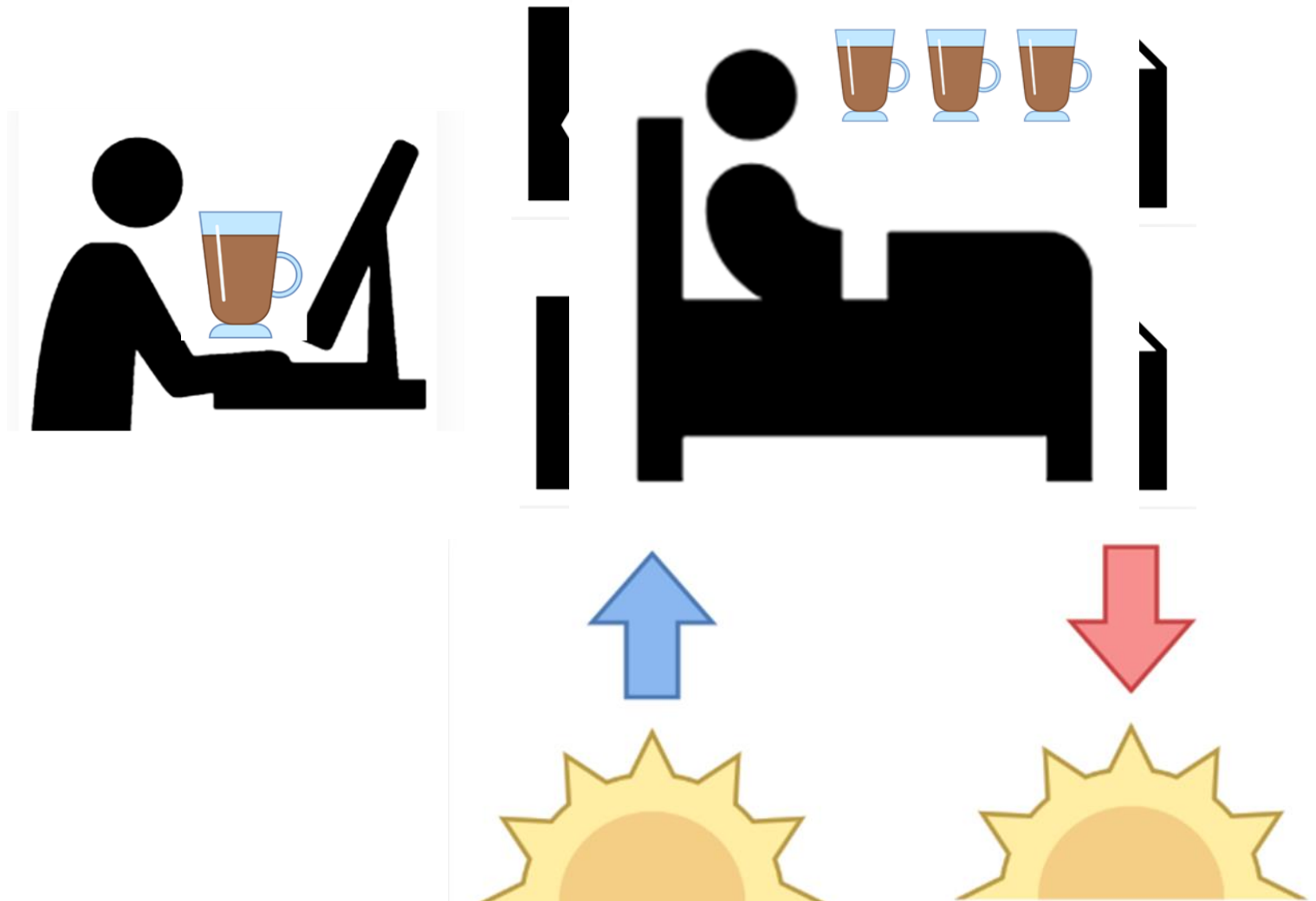
- Specifikáció, követelmények definiálása
 - > Modellek (Use Case), követelmény specifikáció
- Tervezés
 - > Modellek (többi UML modell) és dokumentáció
- Implementálás
 - > Forráskód
- Hibakeresés
 - > Javított forráskód
- Tesztelés
 - > Tesztelt, funkcionálisan működő forráskód
- Üzembehelyezés, telepítés
 - > Telepített, működő alkalmazás
- Üzemeltetés, karbantartás
 - > Alkalmazás (új verziók)

Szoftverfejlesztési dokumentáció

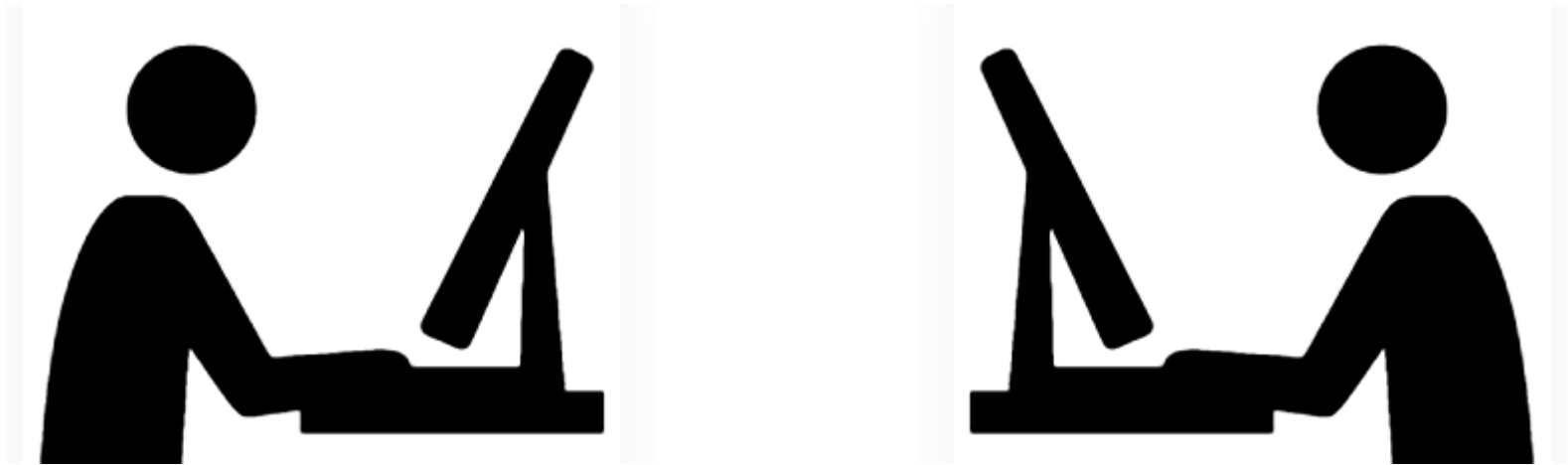
MoonLanding – az ötlet



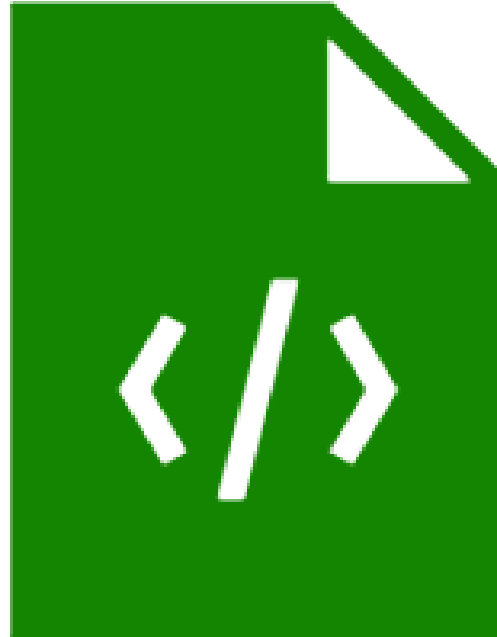
MoonLanding – az implementáció

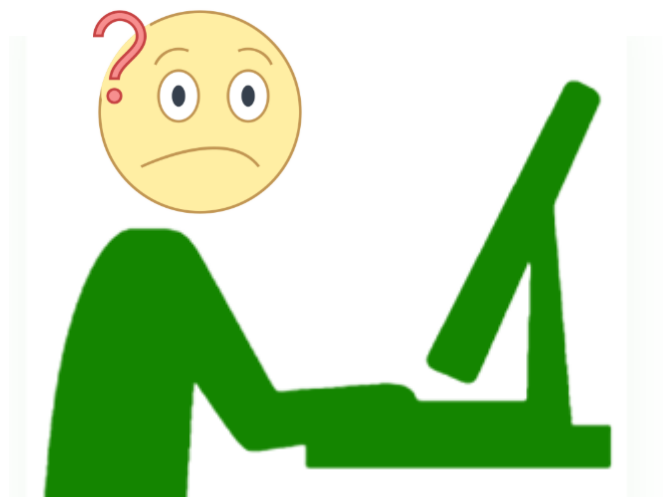


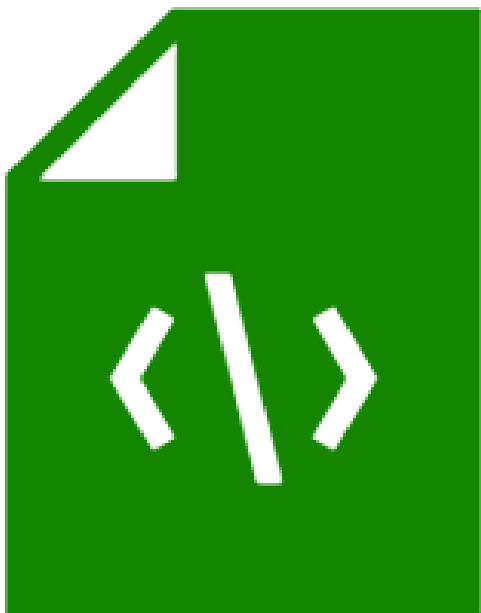
MoonLanding – a társ

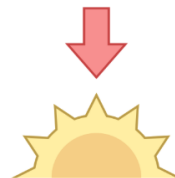






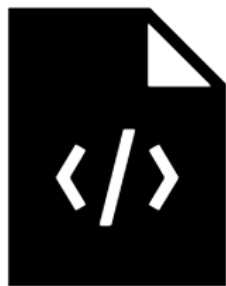
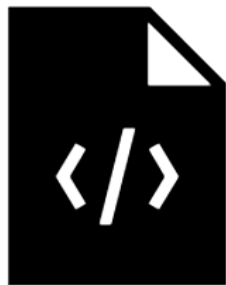


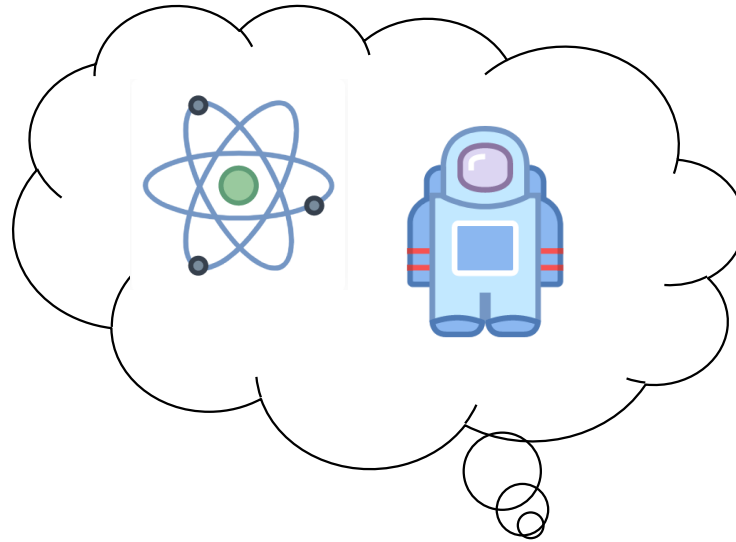


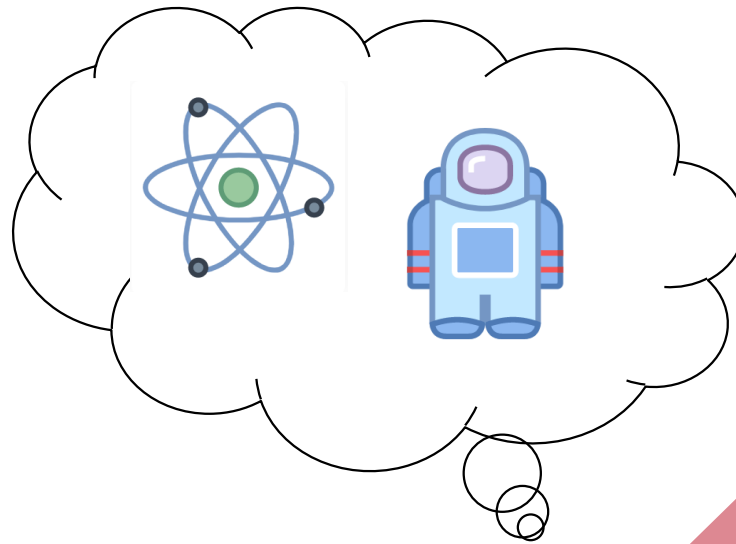


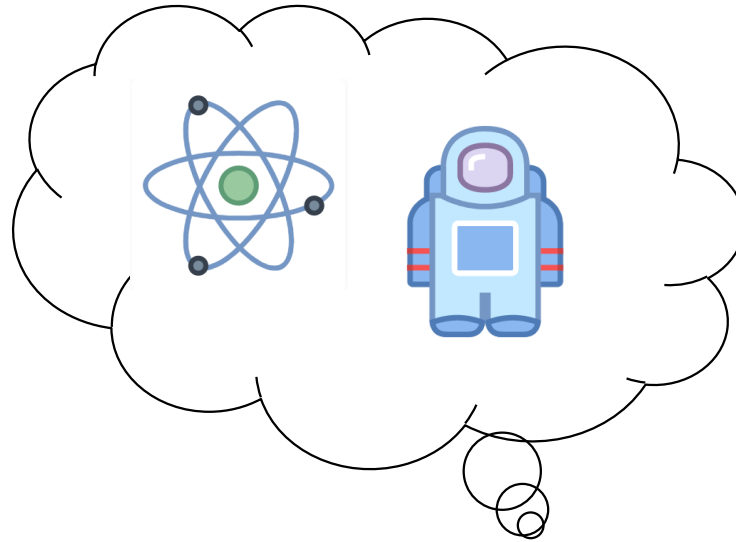








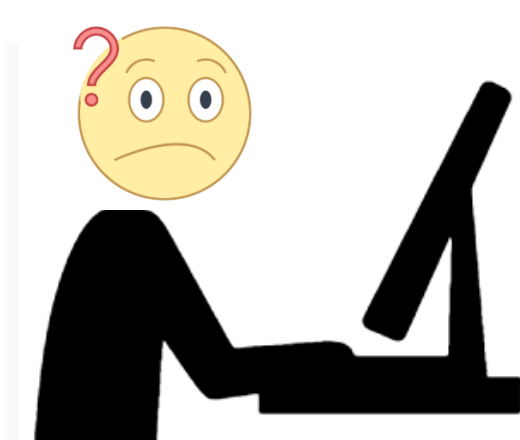
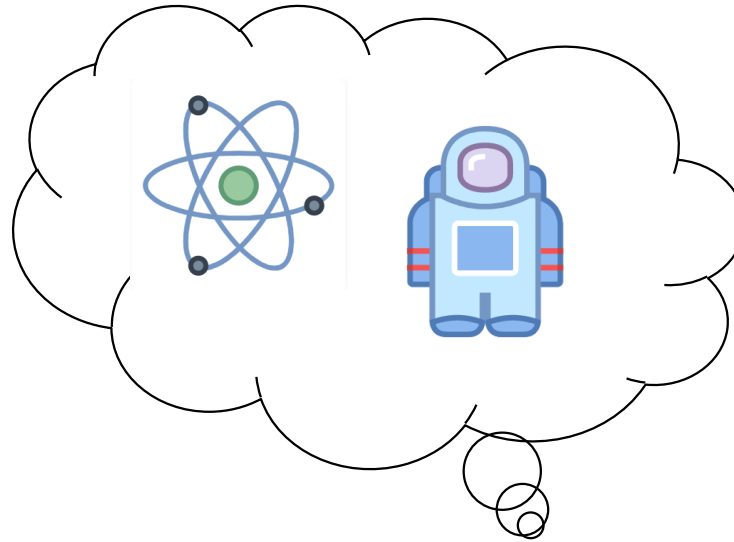












Hogyan lehetett volna ezt elkerülni?

- „Dokumentálással”

Dokumentáció

- **Kód dokumentáció célja: a következő fejlesztő generáció tudjon dolgozni, hibákat javítani, új funkciókat fejleszteni!**
 - Lehetnek egyéb célok: pl a megrendelő fizet érte, törvényi előírás stb.

Dokumentáció formái

- A „dokumentáció” nem feltétlenül kiegészítő szövegesen írott dokumentum
- Alternatívák
 - > „Öndokumentáló” kód: kódolási stílus, beszédes nevek, általános és projekt specifikus tervezési minták stb.
 - > Unit tesztek: egyértelműsítik a komponens határokat, felelősségeket!
 - > UML diagramok – a közös nyelv! Részben generálható is

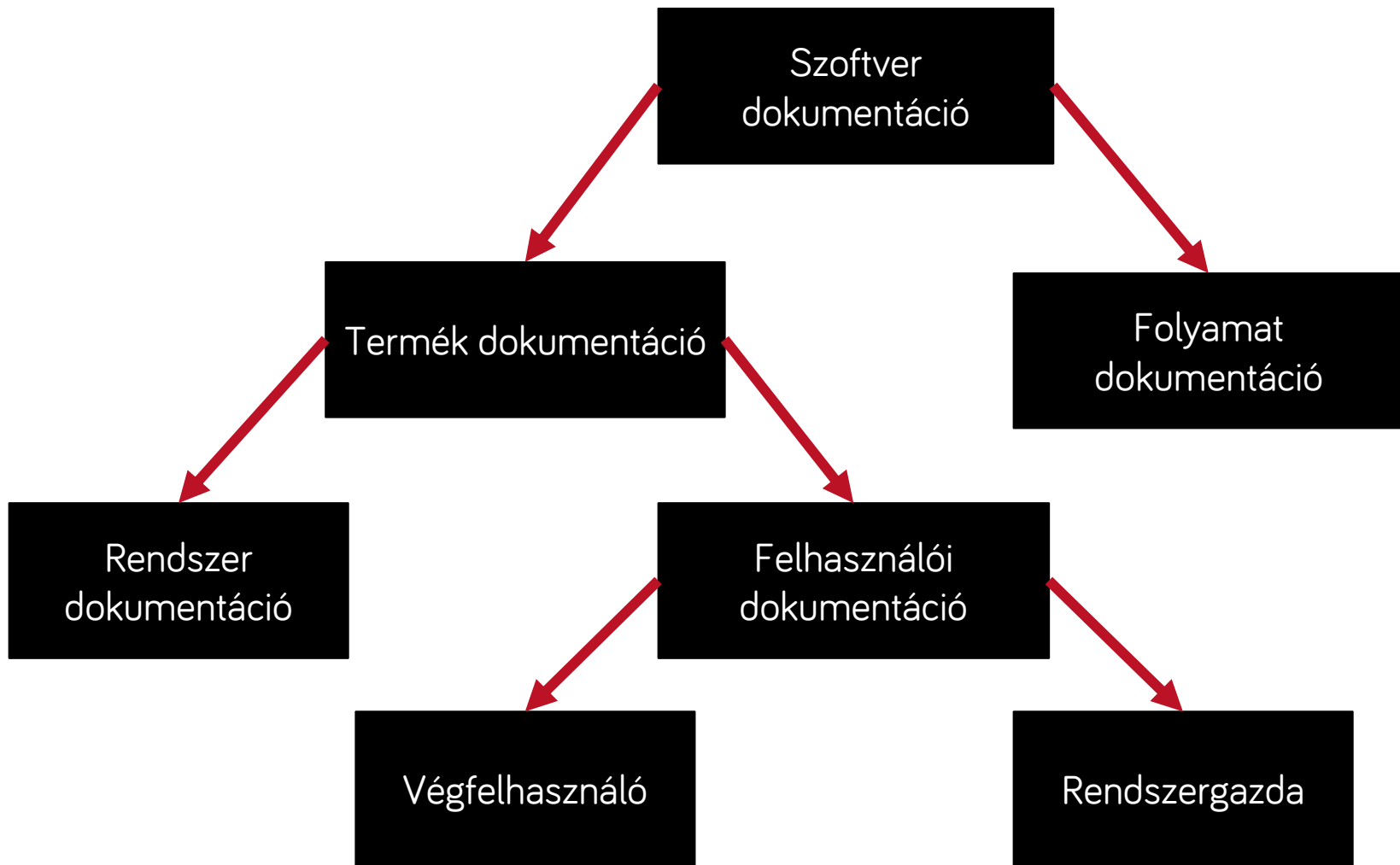
Dokumentáció

- A dokumentálás a szoftverkészítés folyamatát végigkísérő tevékenység.
- Lényegében az összes munkafázishoz kapcsolódik dokumentáció.

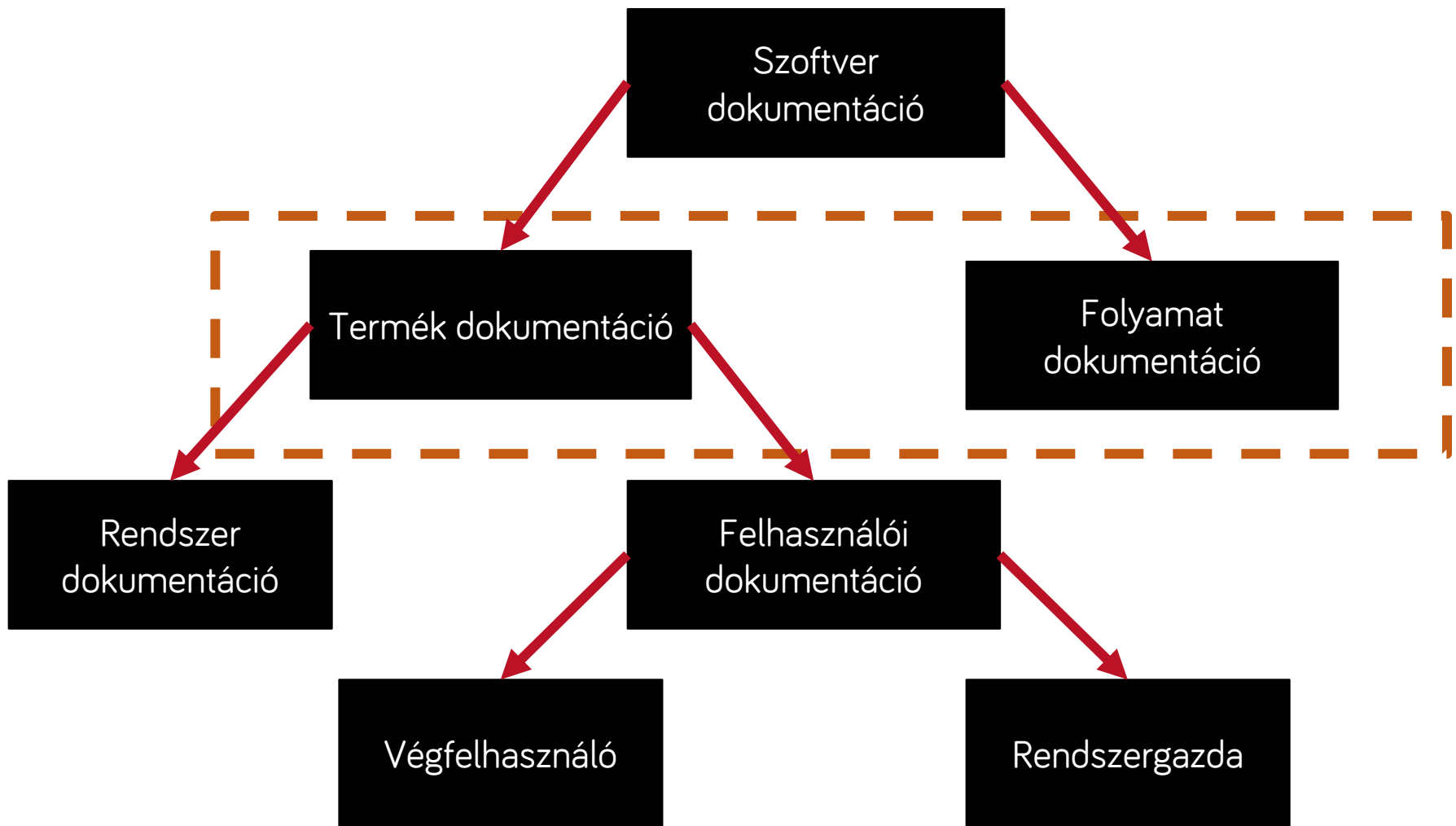
Szoftver

- A szoftver nem csupán programkód!
- Szoftver = programok, dokumentációk, konfigurációk, adatok együttese

Dokumentáció csoportosítása



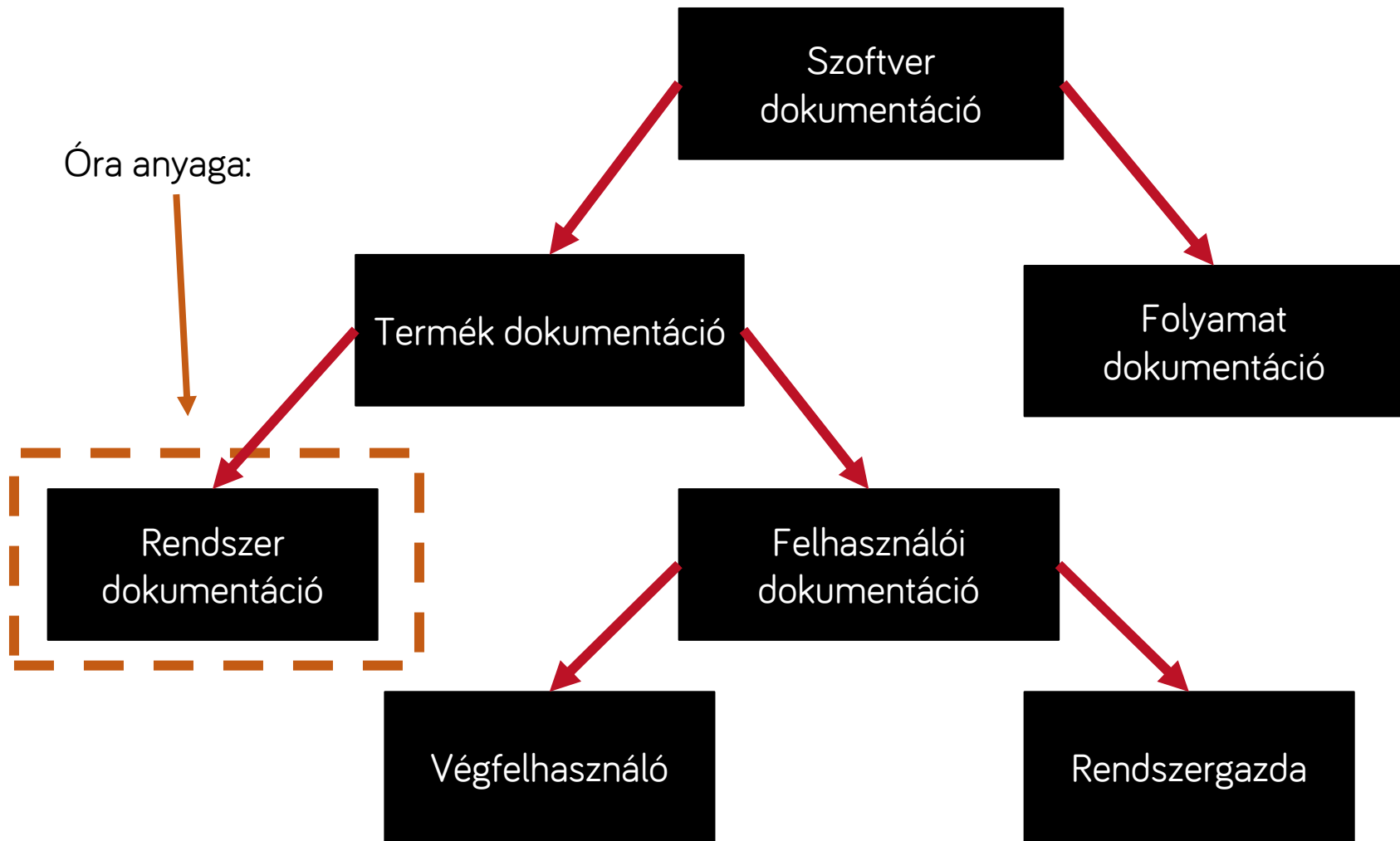
Dokumentáció csoportosítása



Termék vs Folyamat

- Termék dokumentáció:
 - > Leírja a szoftverterméket, ami éppen fejlesztésre kerül
- Folyamat dokumentáció:
 - > A folyamat dokumentáció leírja a folyamatot (pl. mi mennyi ideig fog tartani, projekt terv, meeting jegyzetek)

Dokumentáció csoportosítása



Rendszer dokumentáció

- A termék dokumentáció része
- Egy áttekintést ad a szoftverre
- Ez tartalmazza: FSD, SDD, API doc!
 - > Követelményspecifikációt
 - > Rendszertervet
 - > Forráskód dokumentációt
 - > Tesztelési dokumentációt
 - > Karbantartási dokumentációt

Követelmény specifikáció

- A követelmények definiálása fázishoz tartozik
- A követelmények alapján készítik el
- A gyakorlatban azt foglalja össze, hogy mit kell a rendszernek/modulnak/feature-nek tudnia
- A formátumának tisztának és jól követhetőnek kell lennie
- Elméletben a formája lehet: beszélt nyelven, formalizált leírással, formális matematikai leírással, formális leírónyelvekkel

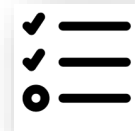


SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA

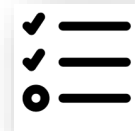
Követelmény specifikáció

Requirements

#	User story title	User story description	Priority	Notes
1	Facebook Integration 	A user wants to sign up via Facebook	Must Have	• We will need to talk to Cassie Owens. • There has also been some research done on this (see Facebook integration prototype)
2	Activity Stream 	A user wants to view the latest updates via the mobile dashboard so that they can get a better understanding of what is in place	Must Have	
3	Post Updates 	A user wants to be able to post status updates on the go	Must Have	The key things we will need to support: • Text status updates • Mentions • Support for images • Smart embedding for YouTube vids
4	API 	A developer wants to integrate with the mobile app so that they can embed the activity stream on their website	Should Have	• We should chat to Team Dyno as they did something similar.



- A tervezés fázisához tartozik
- A fő döntések dokumentálása a szoftver felépítésével kapcsolatban
- Része az architektúra kifejtése
 - > Pl. ha microservice-eket használunk a fejlesztés során, ez mindenképpen említésre kerül itt



- A felhasználói történetek (user story) kifejtése
 - > Az egyes user storyk milyen üzleti folyamatoknak felelnek meg
- A megoldás részletei
 - > Service-ek, modulok, komponensek részletezése és fontosságuknak a tárgyalása
- Diagrammok
 - > Pl. Use case, UML osztálydiagramok, szekvenciadiagramok

Rendszerterv felépítése

- Előszó (célközönség, dokumentum-történet)
- Bevezetés (szoftver célja, helye, szükségessége, előnyei, fejlesztési módszertan)
- Fogalomtár (technikai áttekintés)
- Rendszer architektúra (magas szintű áttekintés, UML csomag-, komponens-, állapotdiagram)
 - > architektúrális minták
 - > funkcionális megfeleltetés
- Adattervezés (adattárolás, formátumok leírása)

Rendszerterv felépítése

- Rendszer tervezés (alacsony szintű áttekintés)
 - > statikus terv (UML osztály-, objektumdiagram)
 - > dinamikus terv (UML állapot-, szekvencia- és aktivitásdiagram)
 - > interfész leírás
 - > felhasznált algoritmusok és minták
- Felhasználói felület (áttekintés, felületi terv)
- Implementációs ajánlások
- Függelék (pl. adatbázis terv, becsült hardver szükségletek)
- Tárgymutató

Forráskód dokumentáció

pl: Javadoc használata



IMPLEMENTÁLÁS

- Az implementációs fázishoz tartozik
- Elmagyarázza, hogy működik a kód
- Szoftverfejlesztőknek szól
- Többek között tartalmazhatja:
 - > A felhasznált keretrendszereket
 - > Adatkötés típusát
 - > Tervezési minták példákkal
 - > Formázási követelmények
 - > API-k leírása

Tesztelési dokumentáció



TESZTELÉS

- Tesztterv
 - > Stratégiák leírása (milyen tesztek lesznek)
 - > Milyen funkciókat kell tesztelni
 - > Milyen módon kell tesztelni őket
 - > Milyen prioritással
 - > Szerepek/felelősségek
- Teszteset dokumentáció
 - > Részletes leírása a tesztesetnek
 - > Mikor verifikálható a teszteset, milyen feltételeknek kell megfelelni
- Teszt ellenőrzőlista

Karbantartási dokumentáció

- Tartalmaznia kell a működés közben felmerült hibákat
- Fejlesztéseket, változtatásokat



ÜZEMELTETÉS, KARBANTARTÁS

Tesztelés

Miért fontos a tesztelés?

- A tesztelés minőségi mutató
 - > Ha nincs tesztelés, nem tudjuk, hogy mennyire jó az alkalmazásunk
 - > Pl. Hogy ne „haljon meg” váratlan felhasználás esetén az alkalmazásunk stb.



Error!

Automatikus vs Manuális

- A tesztelés lehet automatikus vagy manuális
- Manuális tesztelés:
 - Pl. Egy ember végig kattint az alkalmazáson
 - Lehet ad-hoc, forgatókönyv mentén stb.
- Automatikus tesztelés:
 - A gép végzi
 - Nagy rendszereknél szükség van automatikus tesztelésre
 - Mi most ezzel foglalkozunk

Blackbox vs Whitebox

- A programra tekinthetünk úgy, mint egy feketedoboz vagy egy fehérdoz
- Blackbox:
 - Nem látunk bele a programba a tesztelés folyamán, azt tudjuk csak vizsgálni, hogy milyen inputra milyen outputot kapunk
- Whitebox:
 - Belelátunk a programba és speciálisan úgy írjuk meg a tesztet

Tesztípusok

- Komponens tesztek
 - > Unit teszt
 - > Modul teszt
- Integrációs tesztek
- Rendszerteszt
- Átvételi teszt
- Regressziós teszt
- Biztonsági tesztek
- Terhelési tesztek
- Üzemeltetési funkciók és folyamatok tesztje

Unit teszt

- Ez egy komponensteszt, vagyis a rendszer önálló részeit vizsgálja
- A legkisebb szoftveres egységre fókuszál
- A metódusokat kezeli: kimenet-bemenetre fókuszál
 - > Jó-e a visszatérési érték

Unit teszt (példa)

```
public class PhoneValidator
{
    public bool IsValid(string phone)
    {
        return UseSomeRegExToTestPhone(phone);
    }
}
```

```
public void TestPhoneValidator()
{
    string goodPhone = "(123) 555-1212";
    string badPhone = "555 12"

    PhoneValidator validator = new PhoneValidator();

    Assert.IsTrue(validator.IsValid(goodPhone));
    Assert.IsFalse(validator.IsValid(badPhone));
}
```

Modul teszt

- Általában a nem funkcionális részeket teszteli:
 - > Memóriaszivárgás
 - > Sebesség
 - > Szinkronizáció
 - > Bottleneck

Integrációs teszt

- A komponensek közötti interfészeket teszteli
 - > A komponensek közötti kapcsolatot
 - > Az összeillesztéskor keletkező hibát
- Unit tesztek után szokott lenni
- Interfészek definiálásánál kezdődik el a tesztelés
- Hibakeresés nehéz

Rendszerteszt

- Az integrációs teszt után szokott következni
- A rendszerteszt a már kész szoftverterméket teszteli
 - > Hogy megfelel-e a követelményeknek
 - > Megfelel-e a specifikációnak
 - > Minden funkciót ellát-e
 - > Megfelel-e a rendszertervnek
- Feketedobozos teszt (blackbox)
- Van, hogy független cég végzi

Átvételi teszt

- Az átvételi folyamathoz tartozik
 - > Alpha teszt: kész termék tesztje a cégnél, de nem a fejlesztőcsapat által
 - > Béta teszt: végfelhasználók szűk csoportja végzi
 - > Felhasználói átvételi teszt: Ugyanaz, mint a béta teszt, csak sokkal több felhasználóval
 - > Üzemeltetői teszt: Rendszergazdák tesztelik a rendszert

Regressziós teszt

- Korábban megjelent hibát vagy forgatókönyvet tesztel újra
- Tipikusan valamilyen változtatás után végzik: jó-e még a rendszer?

Regressziós teszt

- Amikor fejlesztésnél írunk egy új funkciót, akkor azt szeretnénk, hogy a már meglévő funkciók ne sérüljenek
- Ezt úgy lehet megvalósítani, hogy egy új funkciót akkor fogadunk csak el, ha minden már megírt unit teszt sikeresen lefut
- Ez a regressziós teszt: minden változtatás eredménye az összes unit teszt lefutása

Funkcionális tesztek

- A szoftver működése determinisztikus
- Minél kevesebb teszttel, minél több funkciót fedjünk le

További részletek

- <https://blog.prototypr.io/software-documentation-types-and-best-practices-1726ca595c7f>
- <https://www.altexsoft.com/blog/business/technical-documentation-in-software-development-types-best-practices-and-tools/>



Köszönöm a figyelmet!